Claude 认证架构师 — 基础认证

学习指南（基于官方考试指南）

简介

Claude 认证架构师 — 基础认证证明专家能够在实施基于 Claude 的实际解决方案时做出合理的权衡决策。考试评估 Claude Code、Claude Agent SDK、Claude API 和模型上下文协议（MCP）的基础知识——这些是使用 Claude 构建生产应用的核心技术。

考题基于真实的行业场景：构建客户支持代理系统、设计多代理研究管道、将 Claude Code 集成到 CI/CD、创建开发者生产力工具，以及从非结构化文档中提取结构化数据。

目标候选人

理想候选人是**解决方案架构师**，负责设计和交付基于 Claude 的生产应用。您应具备至少 6 个月的实践经验，涵盖：

- **Claude Agent SDK** — 多代理编排、委派子代理、工具集成、生命周期钩子
- **Claude Code** — CLAUDE.md、MCP 服务器、Agent 技能、规划模式
- **模型上下文协议（MCP）** — 用于后端集成的工具和资源
- **提示工程** — JSON 模式、少样本示例、数据提取模板
- **上下文窗口** — 处理长文档、多代理上下文传递
- **CI/CD 管道** — 自动代码审查、测试生成
- **升级与可靠性** — 错误处理、人在回路

考试格式

参数	值
题型	单选题（4 选 1）
评分	100–1000 分制，及格分数 **720 分**
猜测惩罚	无（请回答每道题！）
场景	从 6 个可能场景中随机抽取 4 个

考试内容：5 个领域

领域	权重
1. 代理架构与编排	**27%**
2. 工具设计与 MCP 集成	**18%**
3. Claude Code 配置与工作流	**20%**
4. 提示工程与结构化输出	**20%**
5. 上下文管理与可靠性	**15%**

考试场景

场景 1：客户支持代理

使用 Claude Agent SDK 构建一个处理退货、账单纠纷和账户问题的代理。该代理使用 MCP 工具（`get_customer`、`lookup_order`、`process_refund`、`escalate_to_human`）。目标是 80% 以上的首次联系解决率和适当的升级处理。

场景 2：使用 Claude Code 生成代码

使用 Claude Code 加速开发：代码生成、重构、调试、文档编写。需要将其与自定义斜杠命令和 CLAUDE.md 配置集成，并了解何时使用规划模式。

场景 3：多代理研究系统

协调代理将任务委派给专业子代理：网络研究、文档分析、综合和报告生成。系统必须生成带有引用的完整报告。

场景 4：开发者生产力工具

代理帮助工程师探索陌生代码库、生成样板代码并自动化日常任务。使用内置工具（Read、Write、Bash、Grep、Glob）和 MCP 服务器。

场景 5：用于持续集成的 Claude Code

将 Claude Code 集成到 CI/CD 管道中，用于自动代码审查、测试生成和拉取请求反馈。提示必须设计为最小化误报。

场景 6：结构化数据提取

系统从非结构化文档中提取信息，用 JSON 模式验证输出，并保持高精度。必须正确处理边缘情况。

官方文档

资源	URL
Claude API — Messages	https://platform.claude.com/docs/en/api/messages
Claude API — Tool Use	https://platform.claude.com/docs/en/build-with-claude/tool-use
Claude API — Message Batches	https://platform.claude.com/docs/en/build-with-claude/message-batches
Claude Agent SDK — 概述	https://platform.claude.com/docs/en/agent-sdk/overview
Claude Agent SDK — Hooks	https://platform.claude.com/docs/en/agent-sdk/hooks
Claude Agent SDK — Subagents	https://platform.claude.com/docs/en/agent-sdk/subagents
Claude Agent SDK — Sessions	https://platform.claude.com/docs/en/agent-sdk/sessions
模型上下文协议（MCP）	https://modelcontextprotocol.io/
MCP — Tools	https://modelcontextprotocol.io/docs/concepts/tools
MCP — Resources	https://modelcontextprotocol.io/docs/concepts/resources
MCP — Servers	https://modelcontextprotocol.io/docs/concepts/servers
Claude Code — 文档	https://code.claude.com/docs/en/overview
Claude Code — CLAUDE.md 与内存	https://code.claude.com/docs/en/memory
Claude Code — 技能（含斜杠命令）	https://code.claude.com/docs/en/skills
Claude Code — Hooks	https://code.claude.com/docs/en/hooks
Claude Code — 子代理	https://code.claude.com/docs/en/sub-agents
Claude Code — MCP 集成	https://code.claude.com/docs/en/mcp
Claude Code — GitHub Actions CI/CD	https://code.claude.com/docs/en/github-actions
Claude Code — GitLab CI/CD	https://code.claude.com/docs/en/gitlab-ci-cd
Claude Code — 无头模式（非交互式）	https://code.claude.com/docs/en/headless
提示工程指南	https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview
扩展思维	https://platform.claude.com/docs/en/build-with-claude/extended-thinking
Anthropic Cookbook（代码示例）	https://github.com/anthropics/anthropic-cookbook

第一部分：理论基础

本部分涵盖成功通过考试所需的全部理论知识。内容按技术和概念组织，而非按考试领域划分——这有助于您对每个主题建立更深入的理解。

第 1 章：Claude API — 模型交互基础

> 文档：Messages API | 提示工程

1.1 API 请求结构

Claude API 遵循请求-响应模型。对 Claude Messages API 的每个请求包含：

```
{
  "model": "claude-sonnet-4-6",
  "max_tokens": 1024,
  "system": "你是一个有帮助的助手。",
  "messages": [
    {"role": "user", "content": "你好！"},
    {"role": "assistant", "content": "你好！"},
    {"role": "user", "content": "你好吗？"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}
}
```

关键字段：

- `model` — 模型选择（`claude-opus-4-6`、`claude-sonnet-4-6`、`claude-haiku-4-5`）
- `max_tokens` — 响应中的最大令牌数
- `system` — 系统提示（定义模型行为）
- `messages` — 对话历史（**必须发送完整历史**以保持连贯性）
- `tools` — 可用工具的定义
- `tool_choice` — 工具选择策略

1.2 消息角色

`messages` 数组使用三种角色：

- `user` — 用户消息
- `assistant` — 模型响应（发送历史时包含）
- `tool` — 工具调用结果（角色未明确设置；以 `tool_result` 内容块的形式出现）

至关重要： 每次 API 请求都必须发送**完整的对话历史**。模型不会在请求之间保持状态——每次调用都是独立的。

1.3 响应中的 `stop_reason` 字段

Claude API 响应包含 `stop_reason`，表示模型停止生成的原因：

值	描述	操作
`"end_turn"`	模型完成了响应	向用户显示结果
`"tool_use"`	模型想要调用工具	执行工具并返回结果
`"max_tokens"`	达到令牌限制	响应被截断；可能需要增加限制
`"stop_sequence"`	遇到停止序列	根据应用程序逻辑处理

对于代理系统，`"tool_use"` 和 `"end_turn"` 最为重要——它们控制代理循环。

1.4 系统提示

系统提示是定义上下文和行为规则的特殊指令。它：

- 不是 `messages` 数组的一部分；通过 `system` 字段单独传递
- 优先于用户消息
- 一次加载，在整个对话中有效
- 用于定义角色、约束和输出格式

考试重点： 系统提示的措辞可能创建意外的工具关联。例如，"始终验证客户"之类的指令可能导致模型过度使用 `get_customer`，即使在不必要的时候。

1.5 上下文窗口

上下文窗口是模型一次可以处理的文本总量（以令牌为单位）。它包括：

- 系统提示
- 完整的消息历史
- 工具定义
- 工具结果

主要上下文窗口问题：

1. **中间丢失效应：** 模型可靠地处理长输入开头和结尾的信息，但可能遗漏中间的细节。缓解方法：将关键信息放在开头或结尾附近。
2. **工具结果积累：** 每次工具调用都会将输出添加到上下文中。如果工具返回 40 多个字段但只需要 5 个，大部分上下文就被浪费了。

3. **渐进式摘要：** 压缩历史时，数值、百分比和日期往往丢失，变成模糊的表达（"大约"、"差不多"、"几个"）。

第 2 章：工具与 `tool_use`

> 文档：Tool Use

2.1 什么是 `tool_use`

`tool_use` 是允许 Claude 调用外部函数的机制。模型不直接运行代码——它生成结构化的工具调用请求；您的代码执行它并返回结果。

2.2 工具定义

每个工具使用 JSON 模式定义：

```
{
  "name": "get_customer",
  "description": "通过邮箱或 ID 查找客户。返回客户档案，包括姓名、邮箱、订单历史和账户状态。使用此工具之前请先于 lookup_order 前验证客户身份。接受邮箱（格式：user@domain.com）或数字 customer_id。",
  "input_schema": {
    "type": "object",
    "properties": {
      "email": {"type": "string", "description": "客户邮箱"},
      "customer_id": {"type": "integer", "description": "数字客户 ID"}
    },
    "required": []
  }
}
```

工具描述的关键要素：

1. **描述是主要选择机制。** LLM 根据工具描述选择工具。最简描述（"获取客户信息"）在工具重叠时会导致错误。
2. **描述中应包含：**
   - 工具的功能和返回内容
   - 输入格式和示例值
   - 边缘情况和约束
   - 何时使用此工具而非类似替代品
3. **避免** 跨工具使用相同或重叠的描述。如果 `analyze_content` 和 `analyze_document` 描述几乎相同，模型会混淆它们。

4. **内置工具与 MCP 工具：** 代理可能更偏好内置工具（Read、Grep）而非功能类似的 MCP 工具。为防止这种情况，需强化 MCP 工具描述——突出内置工具无法提供的具体优势、独特数据或上下文。

2.3 `tool_choice` 参数

`tool_choice` 控制模型如何选择工具：

值	行为	使用时机
`{"type": "auto"}`	模型决定是否调用工具或用文本回答	大多数情况的默认值
`{"type": "any"}`	模型**必须**调用某个工具	需要保证结构化输出时
`{"type": "tool", "name": "extract_metadata"}`	模型**必须**调用特定工具	需要强制第一步/执行顺序时

重要场景：

- `tool_choice: "any"` + 多个提取工具 → 模型选择最佳工具，但仍能获得结构化输出
- 强制选择 → 需要保证特定首个操作（例如，在丰富之前执行 `extract_metadata`）

2.4 结构化输出的 JSON 模式

使用带 JSON 模式的 `tool_use` 是从 Claude 获取结构化输出的**最可靠**方式。它：

- 保证语法上有效的 JSON（无缺失括号，无尾随逗号）
- 强制执行所需结构（必填字段存在）
- **不**保证语义正确性（值仍可能是错误的）

模式设计 — 关键原则：

```json
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
      "enum": ["bug", "feature", "docs", "unclear", "other"]
    },
    "category_detail": {
      "type": ["string", "null"],
      "description": "category = 'other' 或 'unclear' 时的详情"
    },
    "severity": {
      "type": "string",
      "enum": ["critical", "high", "medium", "low"]
    },
    "confidence": {
      "type": "number",
      "minimum": 0,
      "maximum": 1
    },
    "optional_field": {
      "type": ["string", "null"],
      "description": "如果在源中未找到信息则为 Null"
    }
  },
  "required": ["category", "severity"]
}
```

模式设计规则：

1. **必填与可选：** 仅当信息始终可用时才将字段标记为必填。必填字段会促使模型在数据缺失时捏造值。
2. **可空字段：** 对可能缺失的信息使用 `"type": ["string", "null"]`。模型可以返回 `null` 而非产生幻觉。
3. **含 `"other"` 的枚举：** 添加 `"other"` + 详情字符串，避免丢失预定义类别之外的数据。
4. **枚举 `"unclear"`：** 用于模型无法自信选择类别的情况——诚实的 `"unclear"` 优于错误的类别。

2.5 语法错误与语义错误

错误类型	示例	缓解措施
语法	无效 JSON，字段类型错误	使用 JSON 模式的 `tool_use`（可消除）
语义	合计不符，值在错误字段中，幻觉	验证检查，带反馈的重试，自我纠正

第 3 章：Claude Agent SDK — 构建代理系统

> 文档：Agent SDK | Hooks | Subagents | Sessions

3.1 什么是代理循环

代理循环是自主任务执行的核心模式。模型不只是回答——它执行一系列操作：

```
1. 向 Claude 发送带工具的请求
2. 接收响应
3. 检查 stop_reason：
   - "tool_use" -> 执行工具，将结果追加到历史，返回步骤 1
   - "end_turn" -> 任务完成，向用户显示结果
4. 重复直到完成
```

这是模型驱动的方法： Claude 根据上下文和先前的工具结果决定下一个要调用的工具。这与动作顺序固定的硬编码决策树不同。

反模式（避免）：

- 解析助手文本以检测完成（"任务已完成"）
- 使用任意迭代限制（例如 `max_iterations=5`）作为主要停止条件
- 检查助手是否产生文本内容作为完成信号

正确方法： 唯一可靠的完成信号是 `stop_reason == "end_turn"`。

3.2 `AgentDefinition` 配置

`AgentDefinition` 是 Claude Agent SDK 中的代理配置对象：

```
agent = AgentDefinition(
    name="customer_support",
    description="处理客户的退货和订单问题",
    system_prompt="你是一个客户支持代理...",
    allowed_tools=["get_customer", "lookup_order", "process_refund",
"escalate_to_human"],
    # 对于协调代理：
    # allowed_tools=["Task", "get_customer", ...]
)
```

关键参数：

- `name` / `description` — 代理的标识和描述
- `system_prompt` — 带指令的系统提示
- `allowed_tools` — 允许的工具列表（最小权限原则）

3.3 轮辐式：协调代理与子代理

多代理架构通常构建为轮辐式拓扑：

```
          协调代理
        /      |       \
   子代理1   子代理2   子代理3
   （搜索）   （分析）   （综合）
```

协调代理负责：

- 将任务分解为子任务
- 决定需要哪些子代理（动态选择）
- 将工作委派给子代理
- 聚合和验证结果
- 处理错误和重试
- 向用户传达结果

关键原则：子代理有隔离的上下文。

- 子代理**不会**自动继承协调代理的对话历史
- 所有必需的上下文必须在子代理提示中**显式传递**
- 子代理不在调用之间共享内存
- 所有通信通过协调代理流动（用于可观察性和错误控制）

3.4 用于生成子代理的 `Task` 工具

子代理通过 `Task` 工具生成：

```
# 协调代理的 allowedTools 必须包含 "Task"
coordinator_agent = AgentDefinition(
    allowed_tools=["Task", "get_customer"]
)
```

显式上下文传递是强制性的：

```
# 不好：子代理没有上下文
Task: "分析文档"

# 好：提示中包含完整上下文
Task: "分析以下文档。
文档：[完整文档文本]
先前搜索结果：[网络搜索结果]
输出格式要求：[模式]"
```

并行生成： 协调代理可以在一个响应中调用多个 `Task`——子代理并行运行：

```
# 一个协调代理响应包含：
Task 1: "搜索关于 X 的文章"
Task 2: "分析文档 Y"
Task 3: "搜索关于 Z 的文章"
# 三个同时运行
```

3.5 Agent SDK 中的钩子

钩子允许在代理生命周期的特定点进行拦截和转换。

PostToolUse 在工具结果提供给模型之前拦截它：

```
# 示例：规范化来自不同 MCP 工具的日期格式
@hook("PostToolUse")
def normalize_dates(tool_result):
    # 将 Unix 时间戳转换为 ISO 8601
    # 将 "Mar 5, 2025" 转换为 "2025-03-05"
    return normalized_result
```

出站调用拦截钩子阻止违反策略的操作：

```
# 示例：阻止超过 500 美元的退款
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call)
```

关键区别：钩子与提示指令

属性	钩子	提示指令
保证	**确定性**（100%）	**概率性**（>90%，非 100%）
使用时机	关键业务规则、财务操作、合规性	一般偏好、建议、格式
示例	阻止 > $500 的退款	"尝试先解决再升级"

规则： 当失败有财务、法律或安全后果时——使用钩子，而非提示。

3.6 `fork_session` 与会话管理

`--resume <session-name>` 恢复命名会话：

```
claude --resume investigation-auth-bug
```

- 继续之前保存上下文的对话
- 适用于跨多个会话的长期调查
- 风险：如果自上次会话以来文件已更改，工具结果可能已过时

`fork_session` 从共享上下文创建独立分支：

```
代码库调查
         |
    fork_session
    /           \
方法 A：         方法 B：
Redux           Context API
```

- 两个分支继承分支点之前的上下文
- 之后，它们独立分叉
- 适用于比较方法或测试策略

何时启动新会话而非恢复：

- 工具结果已过时（文件已更改）
- 已过很长时间，上下文已降级
- 以"以下是我们发现的简短摘要：..."重新开始，比用旧工具数据恢复更可靠